CS3230 : Design and Analysis of Algorithms (Spring 2014)
Tutorial Set #2
[SOLUTION SKETCHES with Suggestions]

Routine Practice Problems *-- do not turn these in -- but make sure you know how to do them.*

R3. [Fun with Estimation using Bounding of Integrals]

(a) Reproduce by yourself, the proof for estimation of $H(n) = (1/1 + 1/2 + 1/3 + \ldots + 1/n)$, using the method of integration. Namely, reproduce the proof that:

$$\ln(n+1) \le \sum_{k=1}^{n} \frac{1}{k} \le (\ln n) + 1$$

(b) Now, give a similar estimate for $T(n) = (\ln(1) + \ln(2) + \ln(3) + \ldots + \ln(n))$.

Friendly Reminder: Make sure you know how to do this YOURSELF.
(And *not* with the help of the book, the notes, your friend, Google, Wolfram-Alpha, Mathematica, Maple, MatLab, etc)

R4. [Fun with TELESCOPING]

Solve this recurrence using telescoping method: (can you "see" the *telescope*?)

$A_n = A_{n-1} + (n-1)$ for $n>1$, $A_1 = 0$

Answer: A little rewriting of the recurrence (make it more convenient)

$A_n = (n-1) + A_{n-1}$ Now, the telescoping starts…

$= (n-1) + (n-2) + A_{n-2}$ (substitute $A_{n-1} = (n-2) + A_{n-2}$)

$= (n-1) + (n-2) + (n-3) + A_{n-3}$ (substitute $A_{n-2} = (n-3) + A_{n-3}$)

$= (n-1) + (n-2) + (n-3) + (n-4) + A_{n-4}$ (substitute $A_{n-3} = (n-4) + A_{n-4}$)

. . . . . .

$= (n-1) + (n-2) + (n-3) + \ldots + (2) + A_2$ (substitute $A_3 = (2) + A_2$)

$= (n-1) + (n-2) + (n-3) + \ldots + (2) + (1) + A_1$ (substitute $A_2 = (1) + A_1$)

$= (n-1) + (n-2) + (n-3) + \ldots + (2) + (1) + 0$ (substitute $A_1 = 0$)

Complete the rest of the steps yourself. You MUST already know your A, B, C. YES?

Question: Can you see the telescope?

D1. [Two very useful theorems that you can use, repeatedly.]
Let $f(n)$ and $g(n)$ be asymptotically positive functions. Prove that

Lemma 1: If $f(n) = O(F(n))$, $g(n) = O(G(n))$, then $f(n) + g(n) = O(\max(F(n), G(n)))$.

Lemma 2: If $f(n) = O(g(n))$, then $\lg(f(n)) = O(\lg(g(n)))$,
where $\lg(g(n)) \geq 1$ and $f(n) > 1$ for all sufficiently large n.

ANSWER:

Before we prove it, let's see what it means first and how to use it.
First, we try one example.
Let $f(n) = 173n^2 + 61n + 151$, and $F(n) = n^2$, then clearly, $f(n) = O(F(n))$
Let $g(n) = 963n + 106 \lg n + 96$, and $G(n) = n$, then clearly, $g(n) = O(G(n))$
Then *Lemma 1 says*,
$173n^2 + 1024n + 106 \lg n + 247 = f(n) + g(n) = O(\max(n^2, n)) = O(n^2)$,

It also says, that if during the analysis of an algorithm, we analyze two different parts
Part 1 has running time $T(n)$, and
Part 2 has running time $S(n)$,
then total running time is just the *max of the two* (when expressed in asymp. notations).

For example, in algorithm Array-Sum (L02a-Slide-7), initialization and post-processing is $O(1)$, while main loop is $O(n)$, so running time is $O(n)$. So, Lemma says, total time is $O(n)$.

In Selection-Sort (L02a-S45), we analyze (a) the running time of all the Find-Max(A, j) calls separately from (b) the rest of the code which clearly runs in $O(n)$ time. The running time for Part (a) is $O(n^2)$. So, Lemma 1 tells the total running time is $O(n^2)$.

So, Lemma 1 is really a *short cut* for Analysis of Algorithms. *Cool*, right?

Note: Do you see what Lemma 1 is really "*saying*"?
Please continue to learn the "language" of Lemmas and Theorems?

Now, to prove Lemma 1:
First, what are we given: $f(n) = O(F(n))$, $g(n) = O(G(n))$, (1)
And what do we need to prove: $f(n) + g(n) = O(\max(F(n), G(n)))$. (2)

From (1), we know that there is are constants C_1 and n_0, and D_1 and m_0, such that
$f(n) \leq C_1 \cdot F(n)$ [5] whenever $n \geq n_0$,
$g(n) \leq D_1 \cdot G(n)$ [6] whenever $n \geq m_0$,

To make things simpler to understand, let $H(n) = \max\{F(n), G(n)\}$.
Try to visualize the function $H(n)$.

We need to prove that we can find some constant E_1 and p_0, such that
$f(n) + g(n) \leq E_1 H(n)$ whenever $n \geq p_0$,

Then for any $n \geq p_0 = \max\{n_0, m_0\}$, [5] and [6] are both true.
For this value of n, we have two cases:
Case 1: if $F(n) \leq G(n)$, then $H(n) = G(n)$
$f(n) + g(n) \leq C_1 \cdot F(n) + D_1 \cdot G(n) \leq (C_1 + D_1) \cdot G(n) = (C_1 + D_1) \cdot H(n)$
Case 2: if $G(n) \leq F(n)$, then $H(n) = F(n)$
$f(n) + g(n) \leq C_1 \cdot F(n) + D_1 \cdot G(n) \leq (C_1 + D_1) \cdot F(n) = (C_1 + D_1) \cdot H(n)$

Thus, choose $E_1 = (C_1 + D_1)$, and $p_0 = \max\{n_0, m_0\}$, we have proven (2).

Partial Proof of Lemma 2:

First, what are we given: $f(n) = O(g(n))$ (3)

And what do we need to prove: $\lg(f(n)) = O(\lg(g(n)))$ (4)

Note: We are also given "where $\lg(g(n)) \geq 1$ and $f(n) > 1$ for all sufficiently large n" all that is to make the functions behave nicely (not *naughtily*), i.e. non-negative.

Make use of this mathematical fact about strictly increasing functions:

Since $f(n) = \lg n$ is a strictly increasing function, we know that

If $f(n) < g(n)$, then $\lg(f(n)) < \lg(g(n))$,

DIY the rest. (Use Proof of Lemma 1 as a guide.)

D2. Master Theorem.

Use the Master Theorem (whenever possible) to solve for $T(n)$ in each of the following recurrences. Assume that $T(n)$ is a constant for sufficiently small n.

(a) $T(n) = 2T(n/2) + 5$.

$f(n) = 5 = \Theta(1)$, $a=2$, $b=2$. So $n^{\log_b a} = n^{\log_2 2} = n^1 = n$

Since $f(n)=O(n^{1-\varepsilon})$, for $\varepsilon=0.5$. This is **Case 1**. Hence, $T(n)= \Theta(n)$

(b) $T(n) = 2T(n/2) + (2n + 5\lg n)$.

$f(n) = (2n + 5\lg n) = \Theta(n)$, $a=2$, $b=2$. So $n^{\log_b a} = n^{\log_2 2} = n$.

Since $f(n) = \Theta(n) = \Theta(n^{\log_b a})$. This is **Case 2**. Hence, $T(n)= \Theta(n \lg n)$

(c) $T(n) = 2T(n/2) + n^2$.

$f(n) = n^2 = \Theta(n^2)$, $a=2$, $b=2$. So $n^{\log_b a} = n^{\log_2 2} = n$.

So $f(n)= \Omega(n^{1+\varepsilon})$ for $\varepsilon=0.5$ (for example).

Next check regularity condition, i.e. $a f(n/b) \leq c f(n)$, for some constant $c < 1$.

$2(n/2)^2 = (1/2)n^2 \leq c n^2$ for $c = ½$. (the regularity condition is satisfied)

This is **Case 3**. Hence, $T(n) = \Theta(n^2)$

(d) $T(n) = 8T(n/2) + n^2$.

$f(n) = n^2 = \Theta(n^2)$, $a=8$, $b=2$. So $n^{\log_b a} = n^{\log_2 8} = n^3$.

Since $f(n)= n^2 = O(n^{3-\varepsilon})$, for $\varepsilon=0.5$. This is **Case 1**. Hence, $T(n)= \Theta(n^3)$

(e) $T(n) = 3T(n/9) + \sqrt{n}$.

$f(n) = \sqrt{n} = \Theta(n^{0.5})$, $a=3$, $b=9$. So $n^{\log_b a} = n^{\log_9 3} = n^{0.5}$.

Since $f(n) = \sqrt{n} = \Theta(n^{0.5})$. This is **Case 2**. Hence, $T(n)= \Theta(\sqrt{n}(\lg n))$

D3. [Finding Customer Details for ColdCore (pun on Cold Call)]

You have just started an internship in a company called ColdCore. On the second day, you received a random set of m telephone numbers, stored in an array $Q[1..m]$, and a database of n telephone numbers, stored in a sorted array $S[1..n]$ (namely, $S[k] < S[k+1]$ for all k). Your boss wants you to implement the following high-level algorithm:

```
Find-Customer-Details(S, Q)
  for k:=1 to m do {
    determine the position p, such that S[p] = Q[k] using Binary-Search;
        (if Q[k] is not in S[1..n], then set p = –1)
    if (p <> –1) then Print p.    (* you can use p as pointer to get other info  *)
  }
```

(a) What is the running time of the above algorithm? **Answer:** $\Theta(m\ (\lg n))$

(b) Can you give the AA-pattern (algorithm analysis) pattern of the above algorithm.
Answer: Sum-of-Log Pattern.

(c) After working on the problem for one day using the algorithm by your boss, you have a good idea. You request your boss to give the array $Q[1..m]$ as a sorted array. Supposing your boss does that, can you get a *faster* algorithm? What is the running time?

Answer: It depends on the value of m. (small, moderate, BIG)
If ($m < n/(\log n)$), we use the algorithm in **(a)**, which takes $O(m \log n)$ time.
If ($n > m \geq n/(\log n)$), we can perform a merge, which takes $O(n+m) = O(n)$ time.
If ($m \geq n$), we can perform a merge, which takes $O(n+m) = O(m)$ time.

Lesson to be Learnt:
So, what is the lesson to be learnt here?
Often, the best algorithm to be used *depends on the situation*.

In this example, if m is small, just do m binary searches.
But, if m is very big, clearly better to have $Q[1..m]$ sorted, and then do a fast merge.
These two extreme cases are *obvious*.

It is really *between* the two extremes that the *analysis of algorithm* is most useful –
to help to figure out roughly the *crossover point* to *switch* from one algorithm to the other.